

Database Design Document

PhishShield

A Machine Learning-Based Web Application for Phishing URL Detection

1. Introduction

The Database Design Document (DDD) describes the organization, storage, and management of data for the PhishShield web application.

PhishShield allows users to enter a website URL. URL-based features are processed by a trained machine learning model, which classifies the URL as Legitimate or Phishing. The application then displays the prediction along with a security recommendation.

The application uses MySQL as its relational database for storing URL scan information.

2. Objectives of Database Design

The main objectives of the database design are:

- To store URLs submitted by users for analysis.
- To store the prediction generated by the machine learning model.
- To store the security recommendation provided to the user.
- To record the date and time of each URL scan.
- To provide organized and efficient storage and retrieval of scan records.
- To maintain data integrity and consistency.

3. Database Technology

The PhishShield application uses MySQL as its relational database. MySQL provides structured storage and management of application data and is integrated with the Python Flask backend.

- **Database:** MySQL

- **Database Type:** Relational Database
- **Backend:** Python Flask
- **Machine Learning:** Scikit-learn

4. Data Requirements

The main data handled by the PhishShield application are:

1. **Submitted URL** – The website URL entered by the user.
2. **Prediction Result** – The classification generated by the machine learning model.
3. **Security Recommendation** – The recommendation displayed to the user based on the prediction.
4. **Scan Time** – The date and time at which the URL was analyzed.

The URL-based features are processed during URL analysis and provided to the machine learning model for classification.

5. Logical Database Design

The proposed database contains one main entity, **URL_SCAN**.

The URL_SCAN table stores information about each URL analyzed by the PhishShield application.

Attribute	Data Type	Constraint	Description
scan_id	INT	Primary Key, AUTO_INCREMENT	Unique identifier for each scan
url	TEXT	NOT NULL	URL submitted by the user
prediction	VARCHAR(20)	NOT NULL	Classification result: Legitimate or Phishing
recommendation	TEXT	NOT NULL	Security recommendation provided to the user
scan_time	DATETIME	NOT NULL	Date and time of the scan

6. Primary Key

The `scan_id` attribute is the Primary Key (PK) of the `URL_SCAN` table.

It uniquely identifies each URL scan record. The `AUTO_INCREMENT` property allows MySQL to automatically generate a unique identifier for each new scan.

For example:

scan_id	url	prediction
1	https://example.com	Legitimate
2	http://suspicious-site.com	Phishing

7. Entity Description

URL_SCAN

The `URL_SCAN` entity represents one URL analysis performed by the PhishShield application. Each record contains:

- The URL submitted by the user.
- The prediction generated by the machine learning model.
- The security recommendation.
- The date and time of the analysis.

8. Database Schema

The following SQL statement defines the proposed MySQL database and table:

```
CREATE DATABASE phishshield;

USE phishshield;

CREATE TABLE URL_SCAN (
    scan_id INT AUTO_INCREMENT PRIMARY KEY,
    url TEXT NOT NULL,
    prediction VARCHAR(20) NOT NULL,
    recommendation TEXT NOT NULL,
    scan_time DATETIME NOT NULL
);
```

Sample Data

```

INSERT INTO URL_SCAN
(url, prediction, recommendation, scan_time)
VALUES
('https://example.com', 'Legitimate',
 'The URL appears to be safe.', NOW());

INSERT INTO URL_SCAN
(url, prediction, recommendation, scan_time)
VALUES
('http://suspicious-site.com', 'Phishing',
 'Avoid accessing this website.', NOW());

```

9. Entity Relationship Diagram

The logical structure of the database is:

URL_SCAN	
PK	scan_id
	url
	prediction
	recommendation
	scan_time

Figure 1. Entity Relationship Representation of the URL_SCAN Entity

PK = Primary Key.

Since the current design contains one main database entity, no relationship between multiple database tables is required.

10. Data Flow

The data flow of the PhishShield application is:

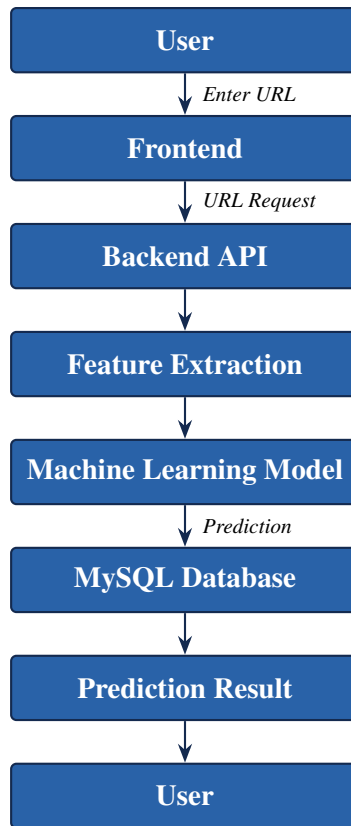


Figure 2. Data Flow of PhishShield

The user enters a URL through the frontend. The backend receives the URL, processes the URL-based features, and passes them to the machine learning model. The resulting prediction and scan information are stored in MySQL, and the result is presented to the user.

11. Data Integrity

The following measures are used to maintain data integrity:

- `scan_id` is defined as the primary key.
- `scan_id` is automatically generated using `AUTO_INCREMENT`.
- The `url` field cannot be empty.
- The `prediction` field cannot be empty.
- The `recommendation` field cannot be empty.
- Appropriate data types are assigned to each attribute.

- Prediction values should be limited to valid classifications such as Legitimate and Phishing.
- Each scan record is uniquely identified by its `scan_id`.

12. Data Security and Privacy

The database should be accessed through the backend application rather than directly by users.

The application should validate user-submitted URLs before processing them. MySQL credentials should be securely stored and should not be exposed in frontend code or publicly accessible files.

Only information required for URL analysis and scan records should be stored. Unnecessary personal or sensitive information should not be collected.

13. Normalization

The `URL_SCAN` table follows basic relational database design principles.

The primary key is `scan_id`. The attributes `url`, `prediction`, `recommendation`, and `scan_time` depend on the corresponding `scan_id`.

The table contains atomic values and does not contain repeating groups. The non-key attributes depend on the primary key.

Therefore, the proposed table structure follows the principles of First Normal Form (1NF), Second Normal Form (2NF), and Third Normal Form (3NF).

14. Database Operations

The Flask backend can perform the following basic operations on the MySQL database:

- **INSERT** – Stores a new URL scan record after analysis.
- **SELECT** – Retrieves stored URL scan records when scan history is required.
- **UPDATE** – Updates an existing scan record when necessary.
- **DELETE** – Deletes a scan record when required.

These operations allow the backend to manage URL scan records stored in MySQL.

15. Future Enhancements

The database can be extended in future versions of PhishShield to support:

- Detailed scan history
- User accounts
- Prediction confidence
- Security analytics
- Statistical reports
- Additional URL features

16. Conclusion

The proposed MySQL database provides a structured and organized method for storing URL scan information and prediction results in PhishShield. The URL_SCAN table maintains the submitted URL, prediction, security recommendation, and scan time.

The design is simple and suitable for the proposed academic web application. MySQL can be integrated with the Flask backend to provide reliable data storage and retrieval while allowing the system to be extended with additional database features in future versions.